

AMD DEVELOPER HACKATHON ACT II — TRACK 1

TokenSage

Hybrid Token-Efficient Routing Agent

Minimize Fireworks AI token spend by routing tasks between a free local vLLM model and a paid large model — without sacrificing accuracy.

PROBLEM

The Token Efficiency Challenge

Local inference is free, but limited

AMD MI300X runs small models at zero cost, but a 3B model lacks accuracy on complex tasks.

Every Fireworks token costs you

The leaderboard ranks by token spend. The winner is whoever uses the fewest Fireworks tokens while meeting the accuracy gate.

The routing tradeoff

Route too aggressively and accuracy drops. Route too conservatively and the token budget burns. You need a smart, adaptive router.

ARCHITECTURE

Routing Pipeline

- 1 Router** Qwen 3B classifies task type and decides LOCAL or BIG with confidence
- 2 Local Solver** Qwen 3B produces a structured JSON answer (thinking, answer, confidence)
- 3 Self-Verification** Re-answers from scratch without seeing the prior output; compares both
- 4 Retry or Escalate** If verification fails, retry locally with chain-of-thought; if still fails, escalate to Fireworks 72B

No Fireworks key? Falls back to local vLLM Qwen 14B automatically.

CORE INNOVATION

Two-Stage LLM Router

Classifies tasks into 5 types, then applies few-shot examples to decide LOCAL vs BIG. Low-confidence LOCAL decisions (<0.3) auto-escalate to BIG.

math

code

reasoning

factual

creative

Key improvements

Few-shot routing

6 labeled examples fix the LOCAL vs BIG decision boundary

Local retry with CoT

Chain-of-thought retry before paying for escalation

Re-answer verification

Solves from scratch — no confirmation bias from seeing the prior answer

Structured JSON

All outputs wrap thinking, answer, confidence for deterministic parsing

KEY TECHNIQUE

Re-Answer Verification

Instead of asking “Is this correct?” — which small models confidently fail at — we ask the model to solve from scratch again.

Naive approach

- Asks the model to judge its own answer — suffers from confirmation bias
- Small models almost never admit mistakes

TokenSage approach

- Re-answers independently without seeing the original output
- Compares both results: if they differ, the original was likely wrong

If the verifier’s confidence is below 0.5, we treat it as a failure regardless of answer match.

DEPLOYMENT

Single Codebase, Two Modes

Dev mode

- No Fireworks key needed
- Big solver: vLLM Qwen 14B (local, free)
- Small model: Qwen 3B
- Built-in sample tasks

Scoring mode

- Fireworks API key injected by platform
- Big solver: Fireworks Qwen 72B
- Reads `/input/tasks.json`, writes `/output/results.json`
- Exact token count logged per task

Same container works in both environments — just set or omit `FIREWORKS_API_KEY`.

PROJECTIONS

Expected Performance

70–
80%

Local Route Rate

<5K

Fireworks Token Budget

75%

Fewer Tokens vs All-Big

The self-verification loop catches most local mistakes before they escalate, keeping accuracy high while staying under the token budget.

STACK

Technology Stack

AMD MI300X · ROCm 7.2 · PyTorch 2.9 · vLLM 0.16 · Qwen2.5-3B · Fireworks AI · Docker · Python 3.11 · AMD AI Notebooks

Boots in under 60 seconds, exits with code 0, runs on AMD MI300X GPU with 80 GB HBM3.

TOKENSAGE

Key Takeaways

Smart, adaptive routing

LLM-based two-stage classifier with confidence thresholding auto-escalates uncertain decisions

Free verification loop

Re-answer from scratch catches 3B mistakes without confirmation bias or Fireworks spend

Zero-cost dev, ready for scoring

Same container runs locally with 14B fallback or on the platform with Fireworks 72B — just set the API key